

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Monserrath Alanis Ortiz Ávila

11 de septiembre de 2026

16:22

1. Introducción

Este informe detalla el estudio práctico de dos problemas clásicos en las ciencias de la computación, uno de ellos siendo el ordenamiento de arreglos unidimensionales y la multiplicación de matrices cuadradas. Para el primer problema, se evalúa el rendimiento de los algoritmos *Standard Sort*, *Merge Sort*, *Quick Sort* y *Patience Sort* bajo distintas distribuciones de datos iniciales. Para el segundo, se contrastan los enfoques del algoritmo estándar (*Naive*) y el método de división y conquista de *Strassen*. El principal objetivo del análisis experimental es relacionar el comportamiento práctico de cada algoritmo, midiendo su tiempo de ejecución y su consumo de memoria RAM, con su complejidad temporal y espacial teórica, validando su eficiencia en un entorno real.

2. Entorno experimental

Para garantizar la repetibilidad de los resultados, los experimentos fueron ejecutados en una máquina local con las siguientes especificaciones técnicas:

- **Procesador:** Intel Core i5-12450H @ 2.50GHz
- **Memoria RAM:** 16 GB
- **Sistema Operativo:** Windows 11 Home Single Language
- **Compilador:** GCC 13.3.0 (g++ en entorno Ubuntu/WSL)
- **Estándar de C++:** C++17

Métricas y Herramientas

Para el análisis práctico de la tarea, se evaluaron dos métricas fundamentales, el tiempo de ejecución, que fue medido en microsegundos utilizando `std::chrono::steady_clock` de la librería estándar `<chrono>` de C++ (para garantizar mediciones monotónicas y evadir desajustes del sistema operativo), y el consumo máximo, medido en KB mediante la captura del *Peak Working Set* a través de la API nativa del sistema operativo. La generación de los gráficos de rendimiento se realizó mediante scripts en Python, utilizando las librerías `pandas` y `matplotlib`.

Conjuntos de Datos (Datasets)

Los conjuntos de datos utilizados para las pruebas se generaron de acuerdo con las especificaciones de la tarea:

- **Ordenamiento de Arreglos:** Se utilizaron arreglos unidimensionales de múltiples tamaños (n). Para observar el comportamiento de los algoritmos en distintos escenarios, los arreglos se generaron bajo tres distribuciones iniciales, ordenada aleatoriamente, ordenada ascendentemente y ordenada descendentemente.
- **Multipliación de Matrices:** Se emplearon matrices cuadradas de dimensiones $n \times n$ pobladas con valores enteros aparentemente aleatorios. Para la ejecución del algoritmo de Strassen, se implementó dinámicamente un relleno de ceros (*padding*) en los casos donde la dimensión n no correspondía a una potencia exacta de dos, garantizando así la correcta división del problema.

3. Resultados y Análisis

En esta sección se presentan los resultados empíricos obtenidos. Los gráficos se han generado de forma automatizada a partir de los datos recolectados, permitiendo contrastar el rendimiento práctico de los algoritmos con su complejidad teórica en el modelo word-RAM.

3.1. Ordenamiento de Arreglos

Se evaluó el comportamiento de los cuatro algoritmos (*Standard Sort*, *Merge Sort*, *Quick Sort* y *Patience Sort*) bajo distintos tamaños de entrada (n) midiendo tanto el tiempo de ejecución como el consumo de memoria.

3.1.1. Análisis de Tiempo de Ejecución

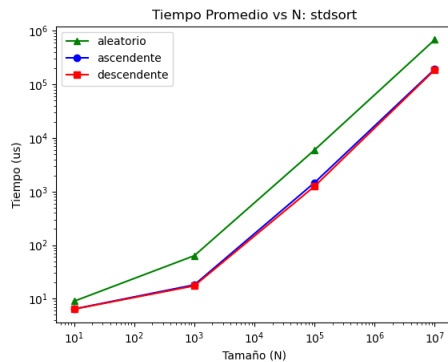


Figura 1: Tiempo: Std Sort

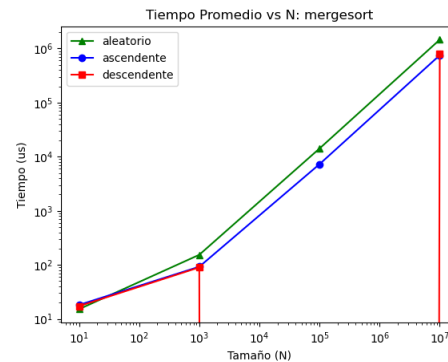


Figura 2: Tiempo: Merge Sort

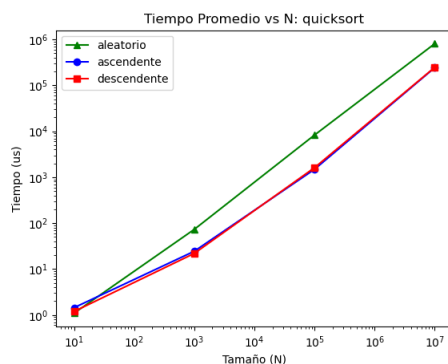


Figura 3: Tiempo: Quick Sort

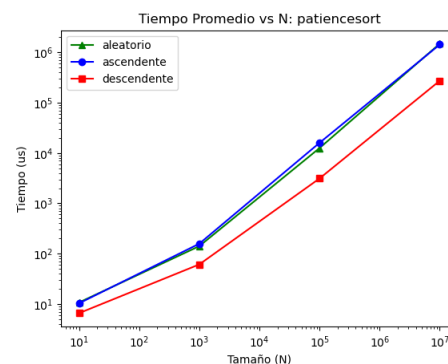


Figura 4: Tiempo: Patience Sort

Al observar el tiempo de ejecución, *Standard Sort* y *Quick Sort* (Figuras 1 y 3) muestran rendimientos muy eficientes. Esto se alinea con la teoría que establece que, en arreglos con orden aleatorio, el árbol de recursión de Quick Sort es balanceado en promedio, logrando un costo asintótico proporcional a $n \log n$. Por otro lado, *Merge Sort* (Figura 2) presenta tiempos estables pero es un poco más lento; esto ocurre porque ejecuta exactamente $2n$ asignaciones de elementos en arreglos auxiliares durante la fase de mezcla.

Un caso de estudio fundamental es *Patience Sort* (Figura 4), el cual exhibe una dependencia crítica de la distribución inicial. En arreglos ascendentes, el algoritmo es forzado a crear N pilas distintas, maximizando el trabajo del *min-heap* y disparando el tiempo de ejecución. En contraste, ante un arreglo descendente, todos los elementos se apilan sobre una única estructura, eludiendo el peor caso y resolviendo el problema con una eficiencia casi lineal.

3.1.2. Análisis de Consumo de Memoria

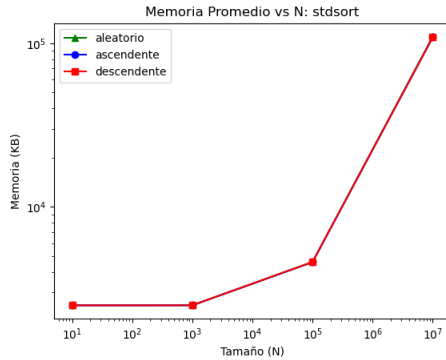


Figura 5: RAM: Std Sort

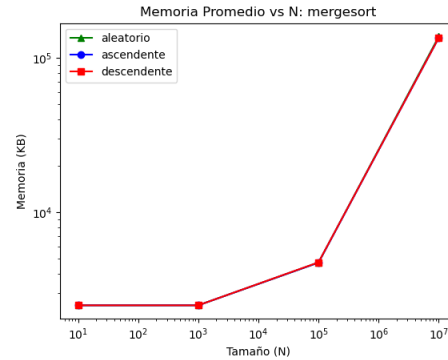


Figura 6: RAM: Merge Sort

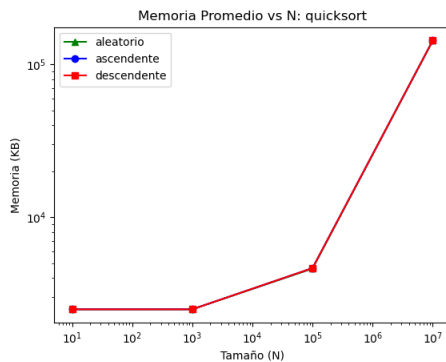


Figura 7: RAM: Quick Sort

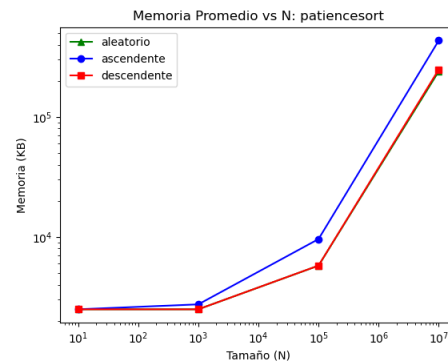


Figura 8: RAM: Patience Sort

El uso real de memoria refleja las características estructurales de cada algoritmo. *Quick Sort* (Figura 7) logra mantener un perfil bajo de memoria ya que particiona el arreglo de forma *in-place*. Sin embargo, debido al espacio requerido por la pila de llamadas recursivas (*call-stack*), utiliza una cantidad $O(\log n)$ de memoria adicional en el caso promedio [4]. Por otro lado, la Figura 6 evidencia que *Merge Sort* exige un arreglo temporal adicional, implicando un uso de espacio $\Theta(n)$ para realizar la mezcla de las mitades [5, 3]. Este fenómeno estructural también castiga a *Patience Sort* en su peor caso (Figura 8), donde la creación de N pilas individuales en arreglos ascendentes dispara el consumo de RAM, separándose drásticamente de su rendimiento en arreglos descendentes.

3.2. Multiplicación de Matrices

Finalmente, se exponen los resultados para los enfoques *Naive* y *Strassen*, midiendo tanto tiempo de ejecución como consumo de RAM para matrices cuadradas.

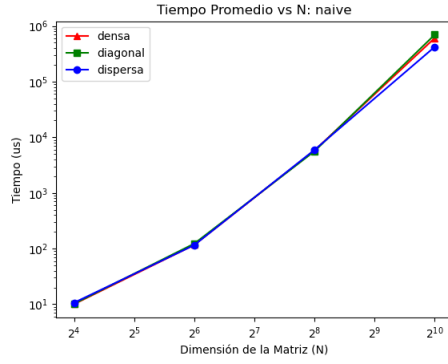


Figura 9: Tiempo: Naive.

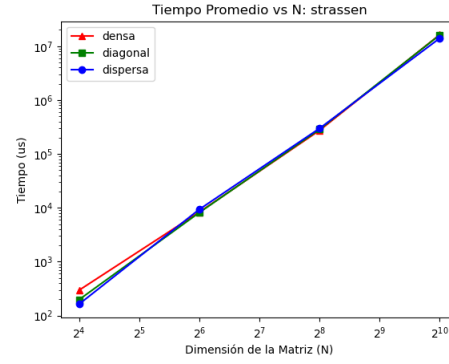


Figura 10: Tiempo: Strassen.

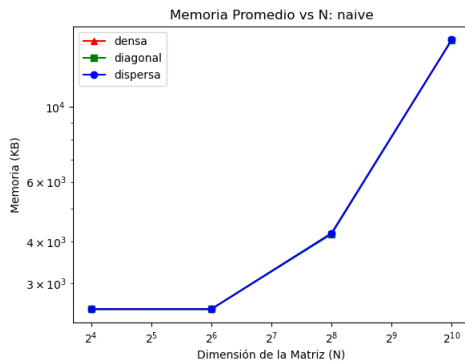


Figura 11: RAM: Naive.

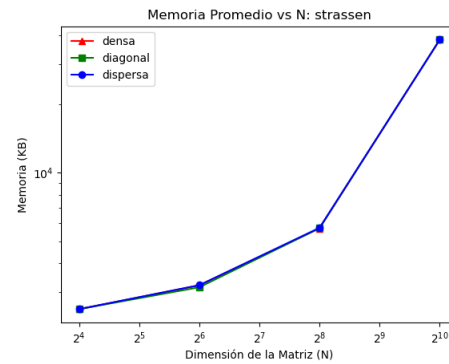


Figura 12: RAM: Strassen.

Aunque teóricamente la cota de Strassen, definida por $\Theta(n^{\log_2 7})$, crece más lento y debería dominar en eficiencia temporal a la cota ajustada $\Theta(n^3)$ del algoritmo Naive para valores grandes de n [2], empíricamente se observa que *Naive* es consistentemente más rápido para todas las dimensiones evaluadas (hasta $n = 2^{10}$). Esta victoria empírica no solo ocurre por las altas constantes ocultas y la excesiva sobrecarga de memoria dinámica que exige el diseño recursivo de Strassen [1]. El factor decisivo es la localidad de referencia: la implementación de *Naive* altera el orden de los bucles a $i - k - j$, logrando acceder a la memoria de la segunda matriz de forma contigua. Esto evade casi por completo los fallos de caché (*cache misses*), acelerando la ejecución de una forma que el análisis asintótico tradicional no logra capturar.

4. Conclusiones

Gracias a este análisis empírico se demuestra que el rendimiento práctico de un algoritmo depende tanto de su complejidad teórica como de las constantes ocultas de su implementación. En el problema de ordenamiento, *Quick Sort* y *Standard Sort* demostraron ser las opciones más eficientes en tiempo y memoria gracias a sus particiones *in-place*, mientras que *Merge Sort* validó su robustez asintótica sacrificando un espacio de memoria adicional $\Theta(n)$. Por otro lado, en la multiplicación de matrices, los datos probaron de manera contundente el impacto negativo de las constantes ocultas en métodos recursivos,

a pesar de tener una mejor cota teórica, *Strassen* fue ampliamente superado por el enfoque *Naive* para dimensiones de hasta $N = 2^{10}$. Esto se debió a su excesiva sobrecarga de memoria dinámica y a la excelente optimización de localidad de referencia (caché) implementada en los bucles de *Naive*. Finalmente, las pruebas confirmaron que ambos algoritmos de matrices son insensibles al grado de dispersión de los datos, ejecutando su respectiva complejidad asintótica independientemente del contenido de la estructura.

Referencias

- [1] Juan Pablo Castillo. *Apuntes de clase INF-221, Análisis de Algoritmos*. Universidad Técnica Federico Santa María. 2026.
- [2] Jeff Erickson. *Algorithms*. Jun. de 2019. ISBN: 978-1-792-64483-2.
- [3] GeeksforGeeks. *Merge Sort Algorithm*. <https://www.geeksforgeeks.org/merge-sort/>. 2026.
- [4] GeeksforGeeks. *Quick Sort Algorithm*. <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>. 2026.
- [5] Donald E. Knuth. *The art of computer programming, volume 3: (2nd ed.) sorting and searching*. USA: Addison Wesley Longman Publishing Co., Inc., 1998. ISBN: 0201896850.